



Universidade de São Paulo

Programação Orientada a Objetos

SSC0504

SPACE BLASTER

RELATÓRIO COMPLETO DO PROJETO

Alunos:

Caio Mendes Laprega

Nº USP: 17018950

Gustavo Furlan Lourenço

Nº USP: 16871332

Tainá Felinto de Araujo

Nº USP: 16922294

Sumário

1. Requisitos.....	2
1.1 Requisitos Funcionais.....	2
1.2 Requisitos não funcionais.....	2
1.3 Requisitos adicionais.....	2
2. Descrição do projeto.....	3
2.1 Visão Geral.....	3
2.2 Arquitetura do projeto.....	3
2.3 Fluxo de telas.....	4
2.4 Sistema de colisão.....	4
2.5 Sistema de pontuação e progressão.....	5
3. Comentários sobre o código.....	5
3.1 Padrões adotados.....	5
3.2 Gerenciamento de memória.....	5
3.3 Decisões de Design.....	5
4. Planos de testes.....	6
4.1 Testes manuais.....	6
5. Resultado dos testes.....	7
6. Procedimentos de Build.....	7
6.1 Pré-requisitos.....	7
6.2 Passos para executar.....	7
6.3 Assets.....	8
7. Problemas.....	8
8. Comentários.....	8
9. Repositório GITHUB.....	9
10. Screenshots.....	9
10.1 Menu Principal.....	9
10.2 Gameplay.....	9
10.3 Game Over.....	10
10.4 High Scores.....	10
10.5 Menu de Instruções.....	11

1. Requisitos

1.1 Requisitos Funcionais

- RF01 — O jogador deve poder mover a nave horizontalmente com as setas do teclado.
- RF02 — O jogador deve poder disparar projéteis com a barra de espaço, com cooldown de 0,25 s entre tiros.
- RF03 — Asteroides devem cair do topo da tela em velocidade e posição aleatórias conforme o nível.
- RF04 — O jogo deve ter 4 níveis progressivos com dificuldade crescente.
- RF05 — Inimigos regulares devem aparecer a partir do nível 2 e disparar projéteis em direção ao jogador.
- RF06 — O nível 4 deve apresentar um Boss com duas fases de comportamento distintas.
- RF07 — O sistema de vidas deve permitir ao jogador receber 3 danos antes do Game Over.
- RF08 — Colisões entre entidades devem ser detectadas e tratadas corretamente (AABB).
- RF09 — O jogo deve manter e exibir a pontuação do jogador em tempo real durante a partida.
- RF10 — As 5 melhores pontuações devem ser salvas em arquivo e exibidas na tela de High Scores.
- RF11 — Ao concluir um nível, deve ser exibida uma tela de transição com bônus e pontuação.
- RF12 — O jogo deve ter tela de menu, instruções, game over e vitória.

1.2 Requisitos Não Funcionais

- RNF01 — O jogo deve rodar a 60 FPS na resolução de 640×480 pixels.
- RNF02 — Sprites PNG e sons WAV/MP3 devem ser carregados dos assets e liberados corretamente da memória.
- RNF03 — O código deve seguir princípios de orientação a objetos com herança e polimorfismo (Entity → Enemy → Boss).
- RNF04 — O projeto deve ser compilável via Gradle e executável com o comando `./gradlew lwjgl3:run`.

1.3 Requisitos Adicionais (implementação própria)

- RA01 — Animação de explosão ao destruir asteroides e o Boss (flash de imagem por 0,18–0,4 s).
- RA02 — Som de tiro ao disparar (Player) e som de explosão ao destruir o Boss.
- RA03 — HUD melhorada com corações de vida, score centralizado, nível no canto esquerdo e barra de HP do Boss com cor dinâmica.
- RA04 — Invencibilidade temporária com animação de piscar (8 Hz) após receber dano.
- RA05 — Contagem mínima de asteroides destruídos para concluir fases 1–3, impedindo vitória por desvio.
- RA06 — Som de vitória ao completar o jogo e som de alternância ao navegar nos menus.
- RA07 — Aumento do spawn rate de meteoritos em níveis avançados para maior desafio.
- RA08 — Penalidade de pontuação ao deixar múltiplos asteroides passarem pela tela consecutivamente, incentivando o foco defensivo.

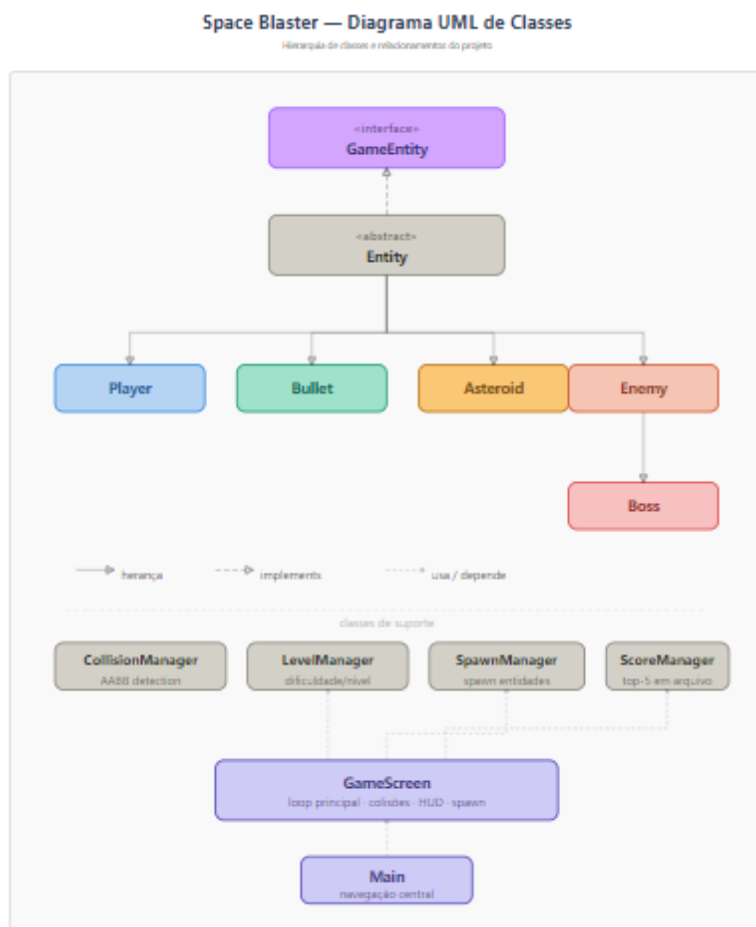
2. Descrição do Projeto

2.1 Visão Geral

Space Blaster é um jogo de nave espacial em estilo shoot 'em up desenvolvido com o framework LibGDX em Java. O jogador controla uma nave que deve destruir asteroides e inimigos ao longo de 4 níveis, culminando em um confronto contra um Boss com duas fases de comportamento.

2.2 Arquitetura do Projeto

O projeto segue uma hierarquia de herança clara, com a interface GameEntity definindo o contrato de toda entidade do jogo, e a classe abstrata Entity fornecendo a implementação base de posição, tamanho e estado de vida.



Classe / Interface	Responsabilidade
GameEntity	Interface: contrato de update(), renderShape(), renderSprite(), getBounds(), isAlive()
Entity	Classe abstrata base: posição, tamanho, alive, destroy(), takeDamage()
Player	Nave do jogador: movimento, disparo com som, sprite, invencibilidade
Bullet	Projétil: sobe (player) ou desce (inimigo), sprite colorido por origem
Asteroid	Asteroide: queda com velocidade aleatória, sprite + animação de explosão
Enemy	Inimigo: movimento horizontal com rebote, tiro periódico, sprite
Boss	Herda Enemy: 2 fases, HP, leque de tiros, sprite maior + explosão com som
GameScreen	Tela principal: loop de jogo, colisões, HUD, spawn, progressão de nível
CollisionManager	Deteção AABB centralizada entre entidades
LevelManager	Parâmetros de dificuldade por nível (velocidade, wave, bônus)
SpawnManager	Controla o spawn de asteroides e inimigos por nível
ScoreManager	Salva e carrega Top-5 pontuações em arquivo scores.txt
Main	Navegação central entre todas as telas (trocarTela)

2.3 Fluxo de Telas

MenuScreen → InstructionsScreen → GameScreen → [LevelCompleteScreen → GameScreen] → WinScreen / GameOverScreen → Main Menu → HighScoresScreen

2.4 Sistema de Colisão

Todas as colisões utilizam AABB (Axis-Aligned Bounding Box) via CollisionManager.check(). São verificadas a cada frame: bala do player vs asteroide, bala do player vs inimigo, bala do player vs Boss, asteroide vs player, bala inimiga vs player e inimigo vs player. Entidades com alive == false são ignoradas automaticamente.

2.5 Sistema de Pontuação e Progressão

- Asteroide destruído: +10 pts
- Inimigo destruído: +25 pts
- Boss destruído: +500 pts
- Bônus de conclusão de nível: +500 / +1000 / +1500 / +2000 pts
- Penalidade: A penalidade é progressiva por sequência de asteroides perdidos.

Nas fases 1–3, o nível só termina quando o jogador destrói o número mínimo de asteroides definido pelo LevelManager, evitando que o jogador vença apenas desviando. Na fase 4, o objetivo é derrotar o Boss. O spawn rate de asteroides aumenta nos níveis posteriores para manter o desafio crescente.

3. Comentários sobre o Código

3.1 Padrões Adotados

O código foi escrito em português para facilitar a leitura pela equipe. Cada classe possui um comentário de cabeçalho explicando sua responsabilidade e, quando relevante, a fase do projeto em que foi implementada (P2/P3). Métodos públicos possuem comentários explicando parâmetros e retornos.

3.2 Gerenciamento de Memória

Todo recurso carregado (Texture, Sound) é liberado via `dispose()` quando a entidade é removida do jogo. O `GameScreen` chama `b.dispose()` e `e.dispose()` em todos os pontos de remoção, tanto nos iteradores quanto nos `removeIf()`, para evitar vazamentos de memória. O `dispose()` do `Player` libera o som de tiro e a textura da nave.

3.3 Decisões de Design

- `Enemy` possui dois construtores: um público (carrega `ufoRed.png`) e um protegido com flag `carregarSprite`. O `Boss` usa o construtor protegido com `false` para não carregar a textura do inimigo comum, já que usa seu próprio `sprite`.
- A animação de explosão é implementada como um flash de imagem (`DURACAO_EXPLOSAO` segundos) que congela a posição da entidade antes de chamá-la de `destroy()`. Isso evita a necessidade de um sistema de partículas.
- O `Boss` chama `destroy()` imediatamente ao HP zerar, mas continua sendo renderizado enquanto `explodindo == true`. O `GameScreen` detecta `!boss.isAlive()` para marcar `bossDestruído` e para de aceitar colisões, mas continua renderizando até a explosão terminar.
- O `SpawnManager` controla se asteroides devem continuar spawnando com base em `asteroidesDestruídos` (fases normais) ou no total da wave (fase Boss), mantendo pressão constante sem depender apenas de contagem de spawns.

- Sons foram adicionados em vários pontos: tiro do player, explosão do boss, vitória ao completar o jogo e feedback sonoro (som discreto) ao alternar opções nos menus para melhorar a experiência do usuário.
- O spawn rate de meteoritos foi aumentado linearmente conforme o nível progride, especialmente nas fases 2 e 3, garantindo que o desafio acompanhe a capacidade crescente do jogador.
- Uma penalidade de pontuação foi implementada, a penalidade é progressiva por sequência de asteroides perdidos. Isso incentiva uma gameplay defensiva e mais envolvente ao invés de apenas desviar passivamente dos asteroides.

4. Plano de Testes

4.1 Testes Manuais

Por se tratar de um jogo em tempo real com elementos gráficos e de entrada de teclado, os testes foram realizados manualmente, verificando cada funcionalidade descrita nos requisitos.

ID	Casos de teste	Procedimento	Result. Esperado
T01	Movimento do player	Pressionar ←/→ durante a partida	Nave move e para na borda
T02	Disparo com cooldown	Segurar ESPAÇO por 2 segundos	Balas saem em intervalos de 0,25s
T03	Colisão bala vs asteroide	Atirar em asteroide	Asteroides explode, +10pts
T04	Vida ao tomar dano	Deixar asteroide colidir com nave	Perde 1 vida, pisca por 2 segundos
T05	Game Over	Perde as 3 vidas	Tela de Game Over com score e som
T06	Progressão de nível	Destruir asteroides suficientes	Tela LevelComplete → próximo nível
T07	Boss — fase 1	Atingir Boss com HP > 10	Boss se move e atira 1 projétil
T08	Boss — fase 2	Reduzir HP do Boss a ≤ 10	Boss acelera e atira leque de 3
T09	Boss — explosão	Destruir o Boss	Explosão com som, tela de vitória
T10	High Scores	Completar partida e ir a High Scores	Score salvo e exibido corretamente
T11	Som de tiro	Disparar com ESPAÇO	Som beep ao atirar
T12	HUD corações	Perder vidas durante a partida	Corações diminuem corretamente

5. Resultado dos testes

ID	Caso de Teste	Resultado	Observações
T01	Movimento do player	☒ Aprovado	Nave limita corretamente nas bordas
T02	Disparo com cooldown	☒ Aprovado	Cooldown de 0,25 s funciona
T03	Colisão bala vs asteroide	☒ Aprovado	Explosão e pontos corretos
T04	Vida ao tomar dano	☒ Aprovado	Invencibilidade e piscar OK
T05	Game Over	☒ Aprovado	Som toca, score exibido
T06	Progressão de nível	☒ Aprovado	Bônus calculado corretamente
T07	Boss — fase 1	☒ Aprovado	Movimento e tiro simples OK
T08	Boss — fase 2	☒ Aprovado	Velocidade e leque corretos
T09	Boss — explosão	☒ Aprovado	Animação e som funcionando
T10	High Scores	☒ Aprovado	Top-5 persistido em scores.txt
T11	Som de tiro	☒ Aprovado	Áudio carregado dos assets
T12	HUD corações	☒ Aprovado	corações diminuem e ficam · ao perder vida

6. Procedimentos de Build

6.1 Pré-requisitos

- JDK 21 (recomendado: Eclipse Temurin 21 — <https://adoptium.net>)
- Git
- IntelliJ IDEA (opcional, mas recomendado) ou qualquer terminal com suporte a Gradle

6.2 Passos para Executar

1. Clonar o repositório:

```
git clone https://github.com/Furlan1/SSC0504-Space-Blaster.git
```

2. Entrar na pasta do projeto:

```
cd SSC0504-Space-Blaster/SSC0504---Space-Blaster-main
```

3. Executar o jogo:

```
./gradlew lwjgl3:run
```


(No Windows sem Git Bash, use: `gradlew.bat lwjgl3:run`)

4. Aguardar o download automático das dependências pelo Gradle na primeira execução.

5. A janela do jogo abrirá automaticamente em 640×480 pixels.

6.3 Assets

Os arquivos de imagem (.png/.jpg) devem estar em `assets/images/` e os sons (.wav) em `assets/sounds/`. A lista completa de assets está documentada no README do repositório.

7. Problemas

- Configuração do JDK no IntelliJ IDEA: o projeto precisou de configuração manual do Run Configuration apontando para `com.spaceblaster.lwjgl3.Lwjgl3Launcher` com `working directory` em `assets/` e JDK 21.
- Vazamento de memória nas Bullets: cada projétil carrega sua própria Texture. Resolvido chamando `b.dispose()` em todos os pontos de remoção da lista (iteradores e `removeIf`).
- Boss carregava textura do Enemy: o Boss chamava `super()` que executava o construtor do Enemy e carregava `ufoRed.png` desnecessariamente. Resolvido com construtor protegido `protected Enemy(..., boolean carregarSprite)`.
- Boss não marcava `bossDestruído` ao iniciar a explosão: o `takeDamage()` passou a chamar `destroy()` imediatamente ao HP zerar, permitindo que o `GameScreen` detectasse `!boss.isAlive()` e marcasse a flag corretamente.
- Working directory incorreto no Gradle via IntelliJ: o `build.gradle` configura `run { workingDir = rootProject.file('assets').path }`, mas o Run Configuration manual precisou ser ajustado manualmente para a subpasta correta.

8. Comentários

O desenvolvimento do Space Blaster foi uma experiência completa de aplicação de orientação a objetos em um contexto de jogo em tempo real. A hierarquia `Entity` → `Enemy` → `Boss` se mostrou eficiente e fácil de estender.

O LibGDX facilitou muito o gerenciamento do loop de jogo, carregamento de assets e detecção de input. O maior desafio foi o gerenciamento correto de memória, garantir que cada Texture e Sound fosse liberado no momento certo exigiu atenção a todos os caminhos de remoção de entidades.

A HUD foi melhorada com corações de vida, score centralizado e barra de HP do Boss com cor dinâmica, tornando a experiência mais visual e intuitiva. Sons de tiro e explosão foram adicionados para dar mais feedback ao jogador.

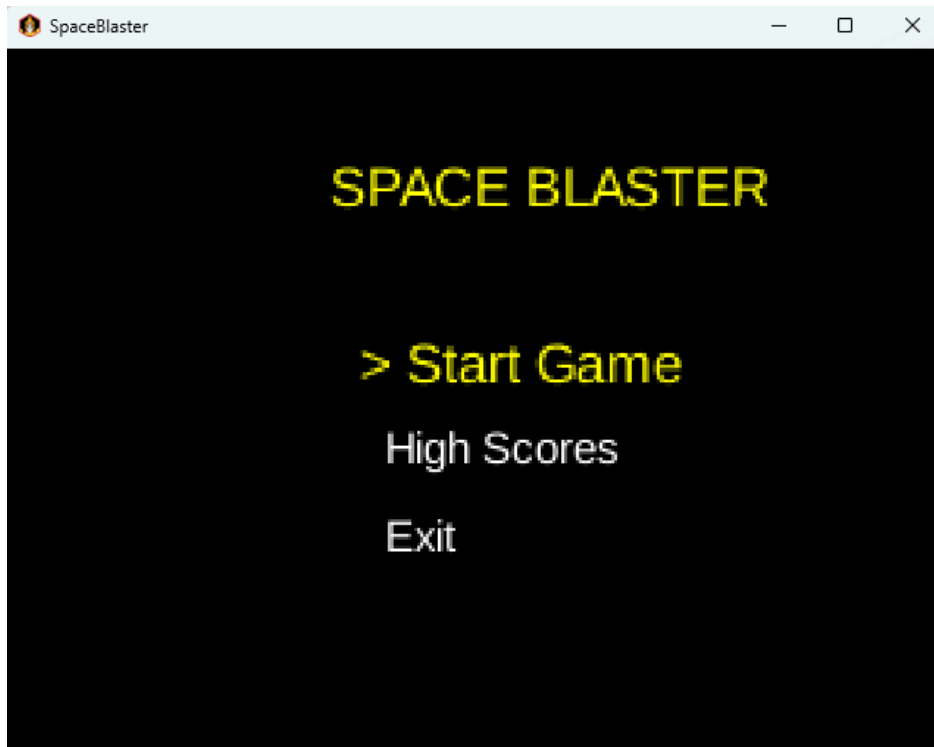
Como melhorias futuras, consideramos: sistema de partículas para explosões mais elaboradas, música de fundo, entrada de nome do jogador nos high scores e suporte a controle de jogo (gamepad).

9. Repositório GITHUB

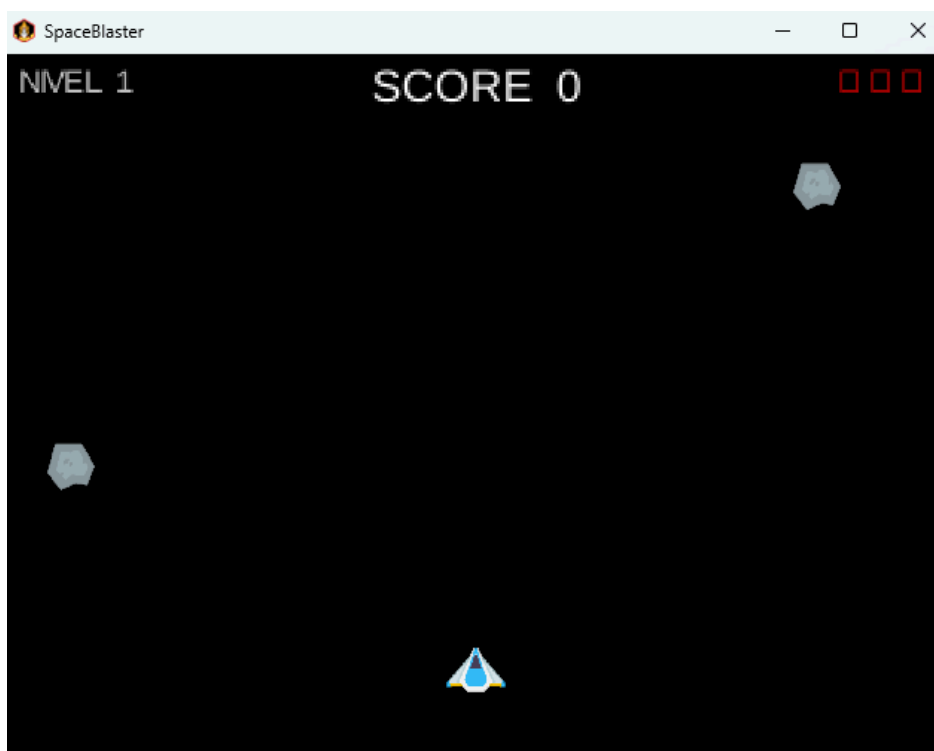
<https://github.com/Furlan1/SSC0504-Space-Blaster/tree/main>

10. Screenshots

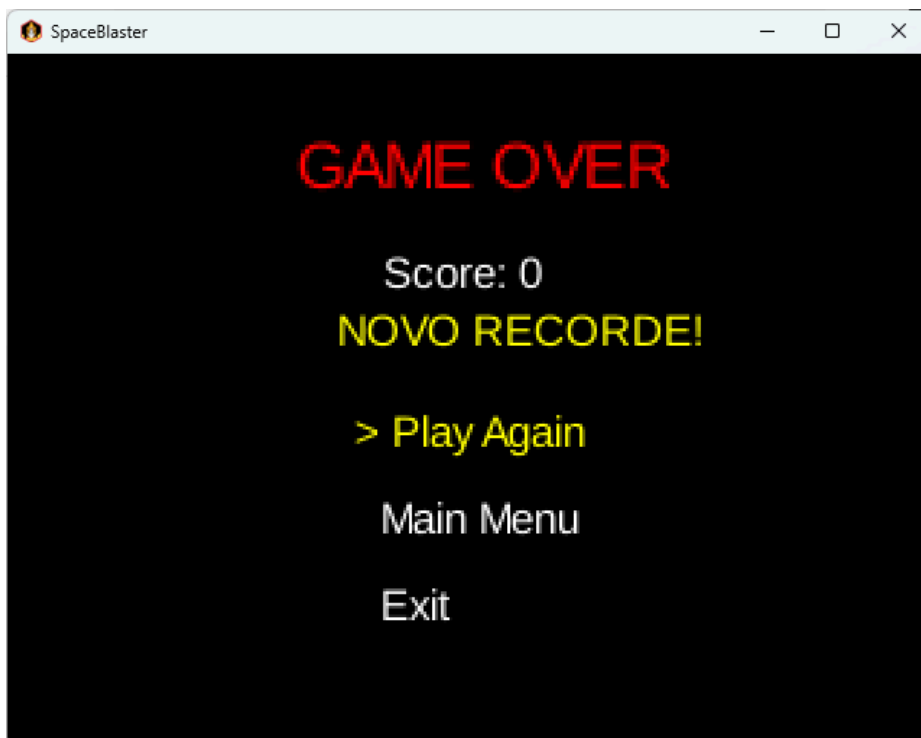
10.1 Menu Principal



10.2 Gameplay



10.3 Game Over



10.4 High Scores

